솔루션 검증 및 사용 사례

NetApp Solutions

NetApp
May 17, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/netapp-solutions/containers/rh-os-n_use_case_pipeline.html on May 17, 2022. Always check docs.netapp.com for the latest.

목차

솔루션 검증 및 사용 사례: NetApp 기반 Red Hat OpenShift 1
영구 스토리지로 Jenkins CI/CD 파이프라인 구축: NetApp과의 Red Hat OpenShift 1
NetApp ONTAP를 사용하여 Red Hat OpenShift에서 멀티 테넌시를 구성합니다 11

솔루션 검증 및 사용 사례: NetApp 기반 Red Hat OpenShift

이 페이지에서 제공되는 예는 NetApp 기반의 Red Hat OpenShift에 대한 솔루션 검증 및 사용 사례입니다.

- "영구 스토리지로 Jenkins CI/CD 파이프라인을 구축합니다"
- "NetApp과 함께 Red Hat OpenShift에서 멀티 테넌시를 구성합니다"
- "NetApp ONTAP 기반의 Red Hat OpenShift 가상화"
- "NetApp OpenShift를 사용한 Kubernetes용 고급 클러스터 관리"

"다음: 비디오 및 데모."

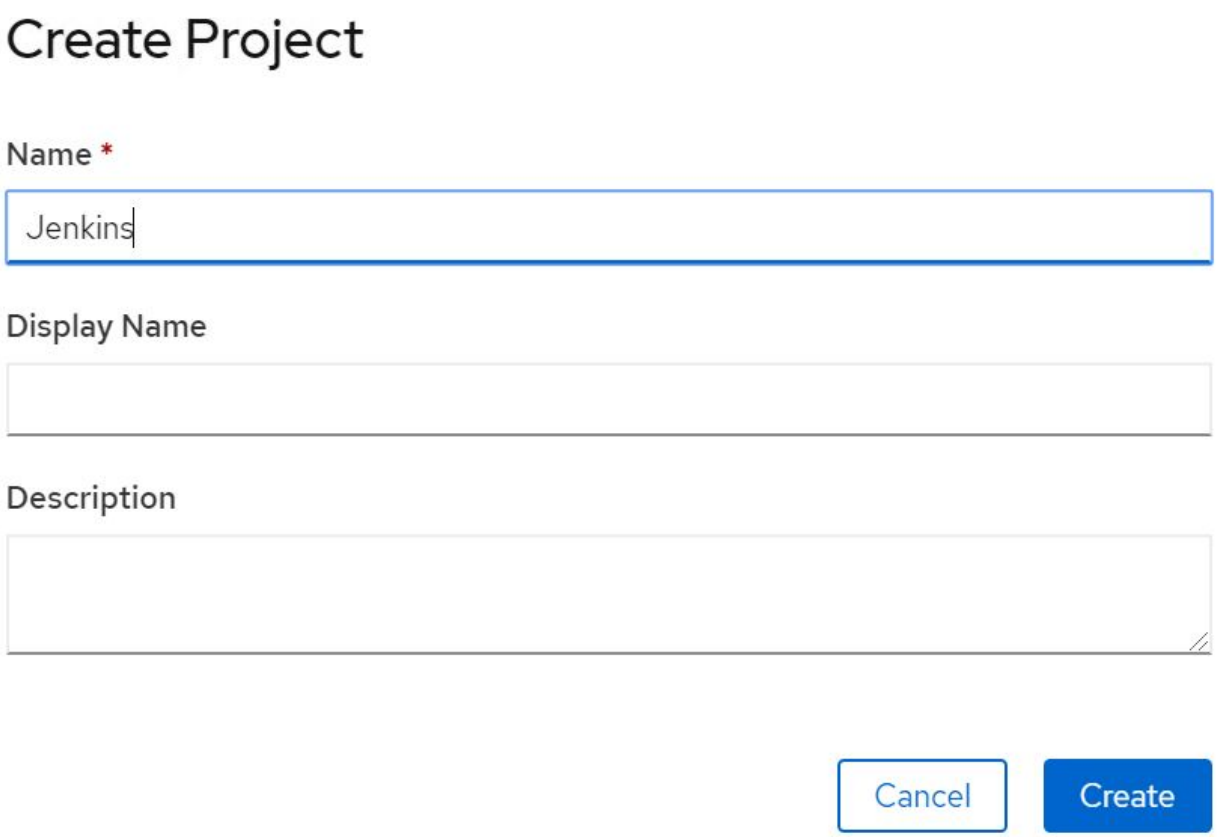
영구 스토리지로 Jenkins CI/CD 파이프라인 구축: NetApp과의 Red Hat OpenShift

이 섹션에서는 Jenkins와 지속적인 통합/지속적인 공급 또는 배포(CI/CD) 파이프라인을 구축하여 솔루션 운영을 검증하는 단계를 제공합니다.

Jenkins 배포에 필요한 리소스를 생성합니다

Jenkins 응용 프로그램 배포에 필요한 리소스를 생성하려면 다음 단계를 수행하십시오.

1. Jenkins라는 새 프로젝트를 만듭니다.



2. 이 예에서는 영구 스토리지에 Jenkins를 구축했습니다. Jenkins 빌드를 지원하려면 PVC를 만듭니다. 저장소 > 영구 볼륨 클레임 으로 이동하고 영구 볼륨 클레임 생성 을 클릭합니다. 생성된 스토리지 클래스를 선택하고 영구 볼륨 클레임 이름이 Jenkins 인지 확인하고 적절한 크기 및 액세스 모드를 선택한 다음 만들기 를 클릭합니다.

Project: jenkins

Create Persistent Volume Claim

Edit YAML

Storage Class

SC basic

Storage class for the new claim.

Persistent Volume Claim Name *

jenkins

A unique name for the storage claim within the project.

Access Mode *

Single User (RWO) Shared Access (RWX) Read Only (ROX)

Permissions to the mounted drive.

Size *

100 GiB

Desired storage capacity.

Use label selectors to request storage

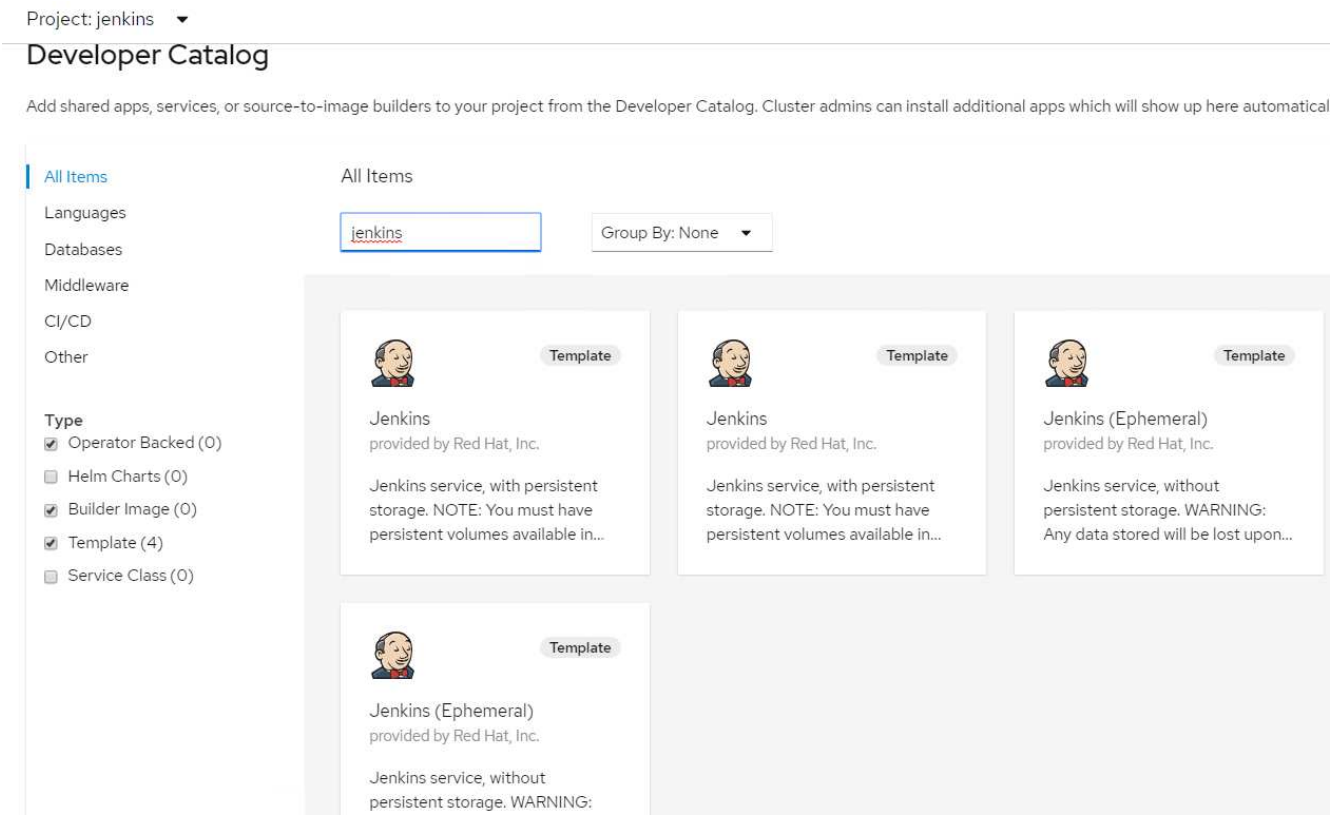
Use label selectors to define how storage is created.

Create Cancel

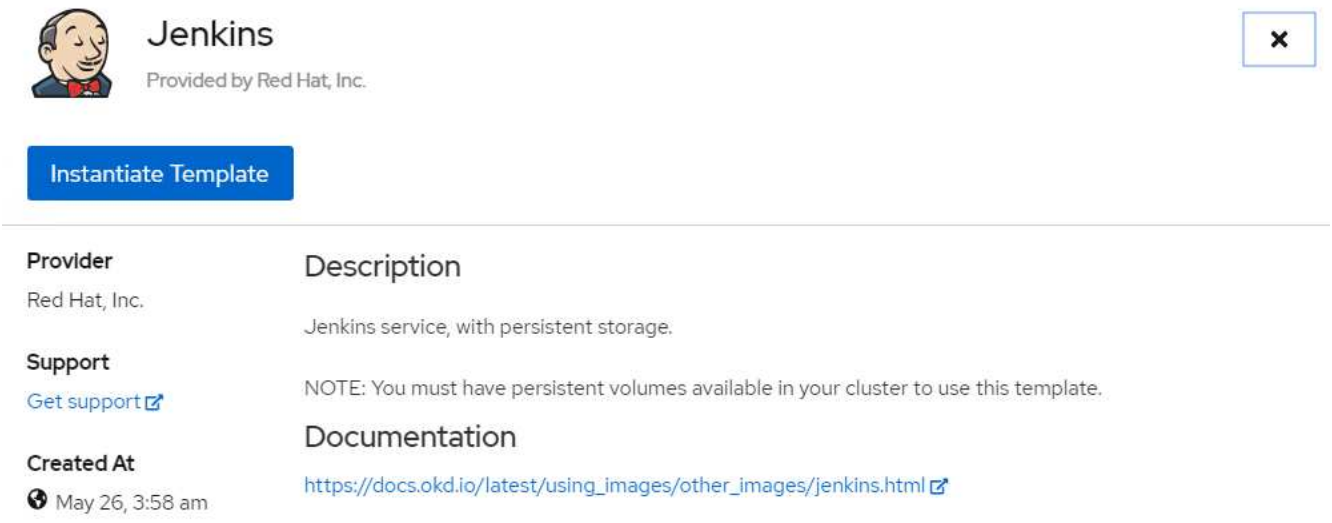
영구 스토리지로 **Jenkins**를 배포합니다

영구 스토리지로 Jenkins를 배포하려면 다음 단계를 수행하십시오.

1. 왼쪽 위 모서리에서 역할을 관리자 에서 개발자 로 변경합니다. Add(추가) 를 클릭하고 Catalog(카탈로그) 에서 선택합니다. 키워드별 필터 표시줄에서 Jenkins를 검색합니다. 영구 저장소를 사용하는 Jenkins Service 를 선택합니다.



2. "인스턴스화 템플릿"을 클릭합니다.



3. 기본적으로 Jenkins 응용 프로그램의 세부 정보가 채워집니다. 요구 사항에 따라 매개 변수를 수정하고 생성 을 클릭합니다. 이 프로세스는 OpenShift에서 Jenkins를 지원하는 데 필요한 모든 리소스를 생성합니다.

Instantiate Template

Namespace *

PR jenkins

Jenkins Service Name

jenkins

The name of the OpenShift Service exposed for the Jenkins container.

Jenkins JNLP Service Name

jenkins-jnlp

The name of the service used for master/slave communication.

Enable OAuth in Jenkins

true

Whether to enable OAuth OpenShift integration. If false, the static account 'admin' will be initialized with the password 'password'.

Memory Limit

1Gi

Maximum amount of memory the container can use.

Volume Capacity *

50Gi

Volume space available for data, e.g. 512Mi, 2Gi.

Jenkins ImageStream Namespace

openshift

The OpenShift Namespace where the Jenkins ImageStream resides.

Disable memory intensive administrative monitors

false

Whether to perform memory intensive, possibly slow, synchronization with the Jenkins Update Center on start. If true, the Jenkins core update monitor and site warnings monitor are disabled.

Jenkins ImageStreamTag

jenkins:2

Name of the ImageStreamTag to be used for the Jenkins image.

Fatal Error Log File

false

When a fatal error occurs, an error log is created with information and the state obtained at the time of the fatal error.

Allows use of Jenkins Update Center repository with invalid SSL certificate

false

Whether to allow use of a Jenkins Update Center that uses invalid certificate (self-signed, unknown CA). If any value other than 'false', certificate check is bypassed. By default, certificate check is enforced.

Create Cancel

Jenkins

INSTANT-APP JENKINS

View documentation Get support

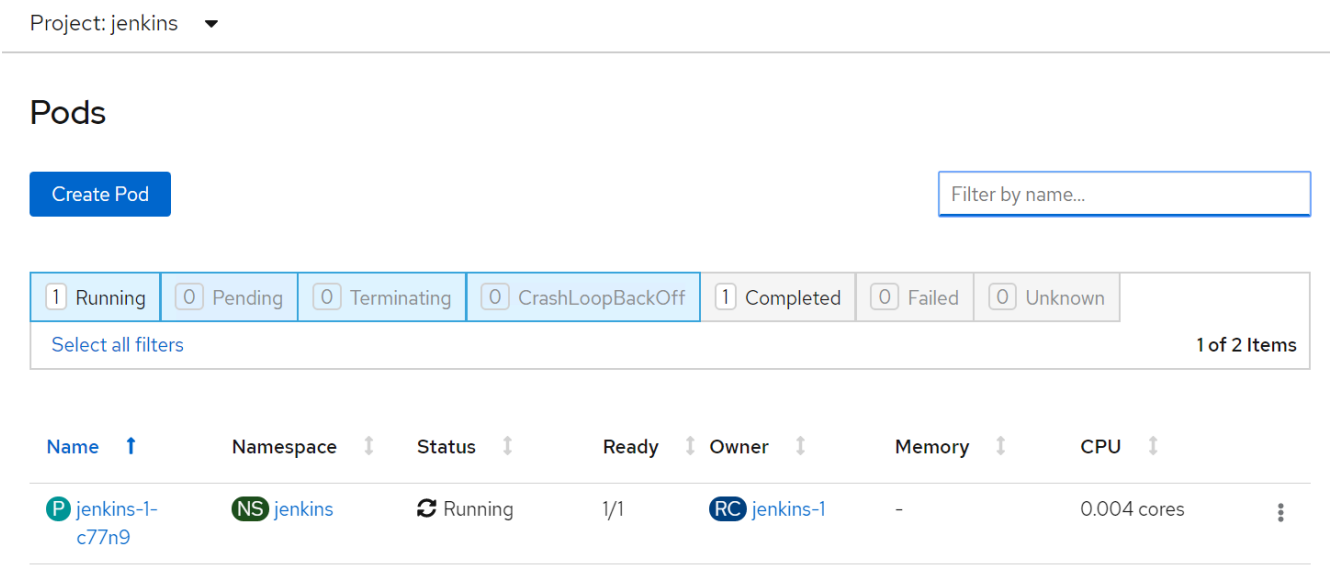
Jenkins service, with persistent storage.

NOTE: You must have persistent volumes available in your cluster to use this template.

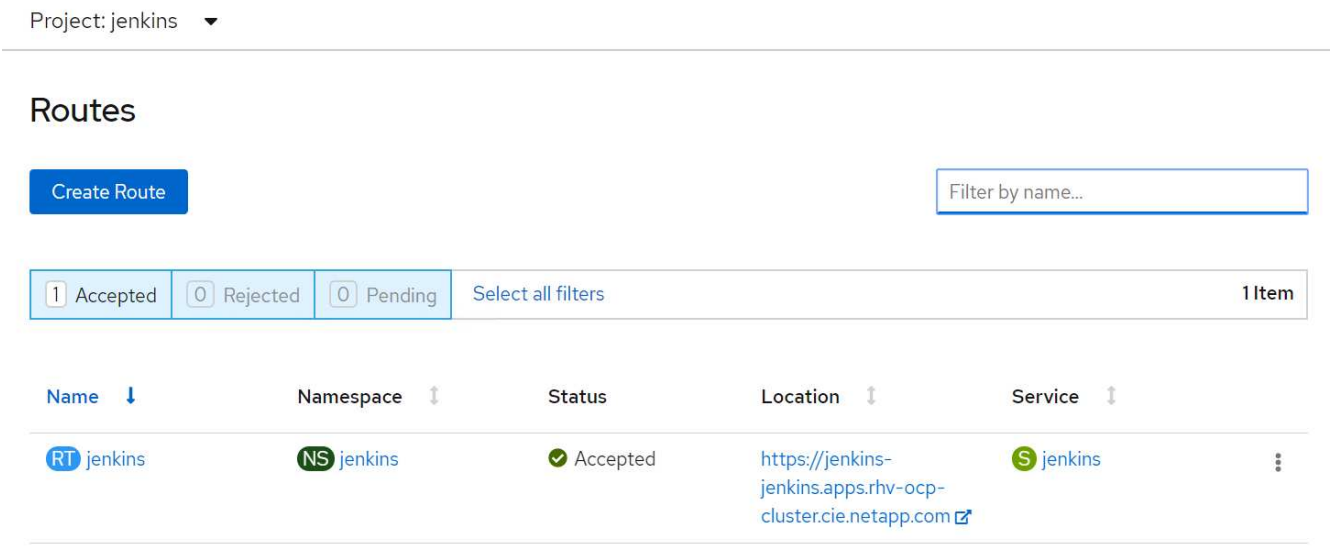
The following resources will be created:

- DeploymentConfig
- PersistentVolumeClaim
- RoleBinding
- Route
- Service
- ServiceAccount

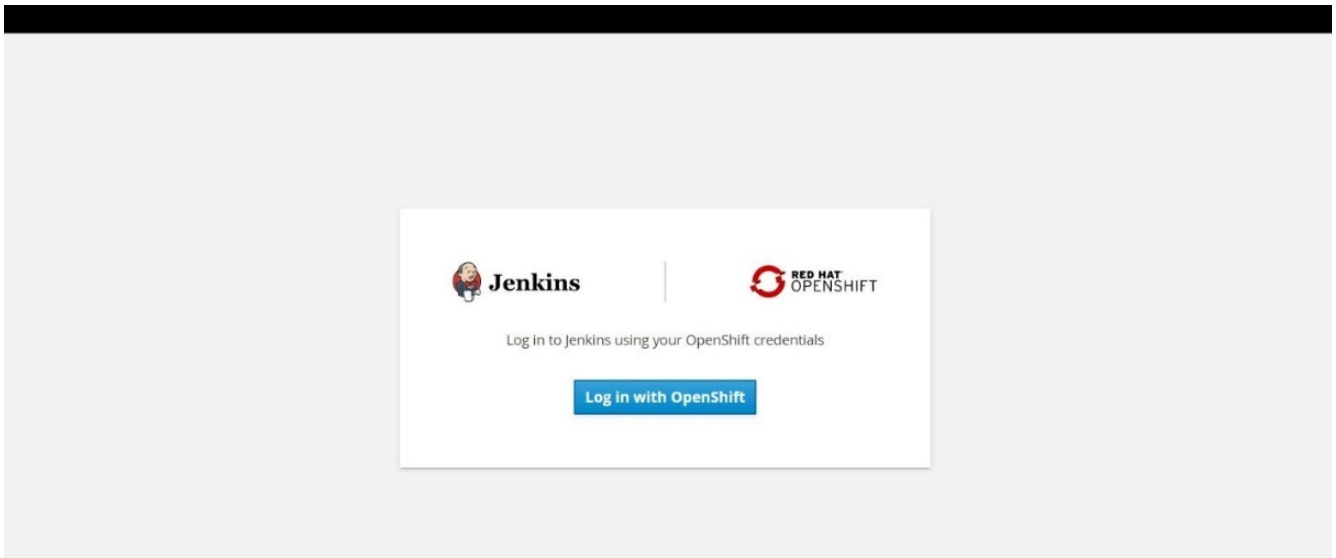
4. Jenkins Pod는 Ready 상태로 들어가려면 약 10~12분이 걸립니다.



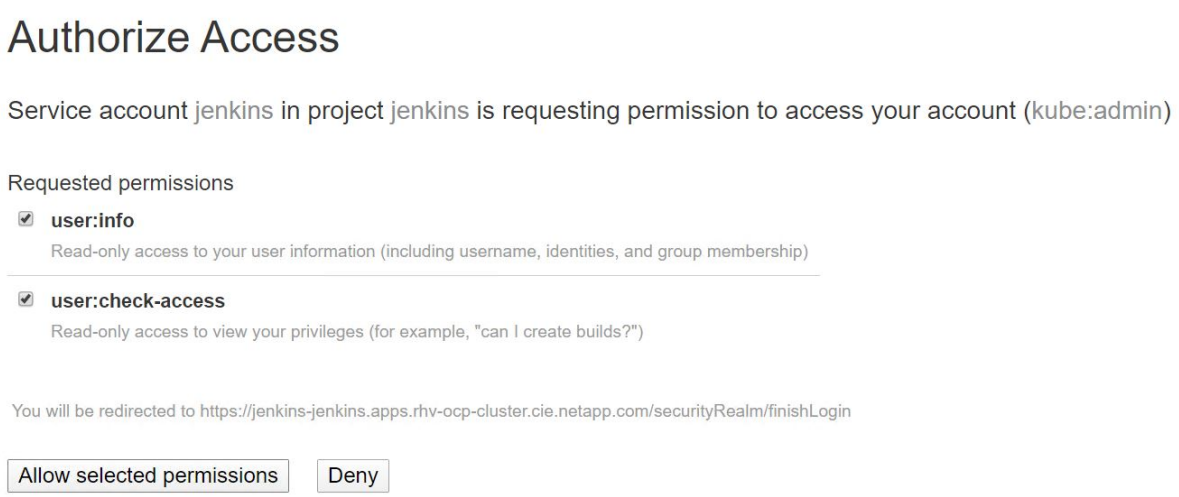
5. 포드가 인스턴스화된 후 네트워킹 > 라우트로 이동합니다. Jenkins 웹 페이지를 열려면 Jenkins 루트에 제공된 URL을 클릭합니다.



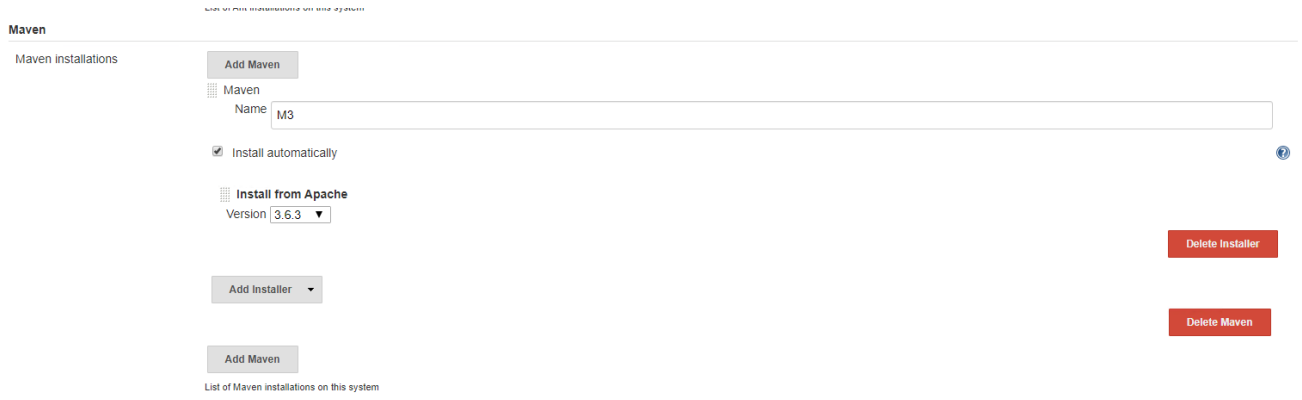
6. Jenkins 앱을 만드는 동안 OpenShift OAuth가 사용되었기 때문에 OpenShift로 로그인 을 클릭합니다.



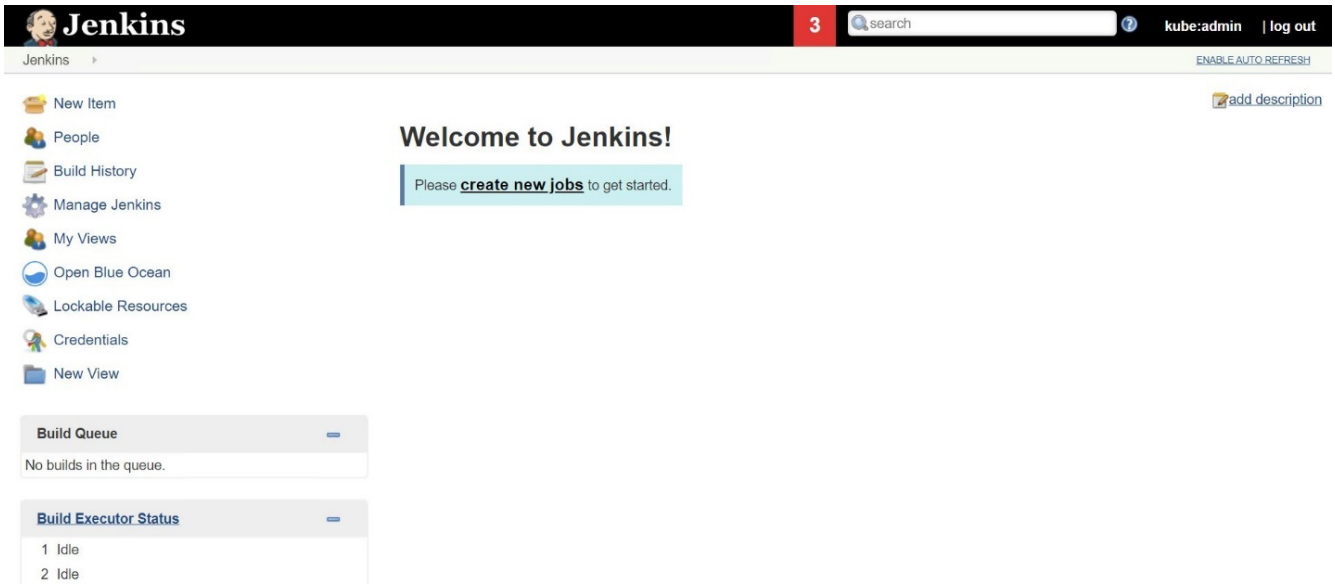
7. Jenkins 서비스 계정을 인증하여 OpenShift 사용자에게 액세스합니다.



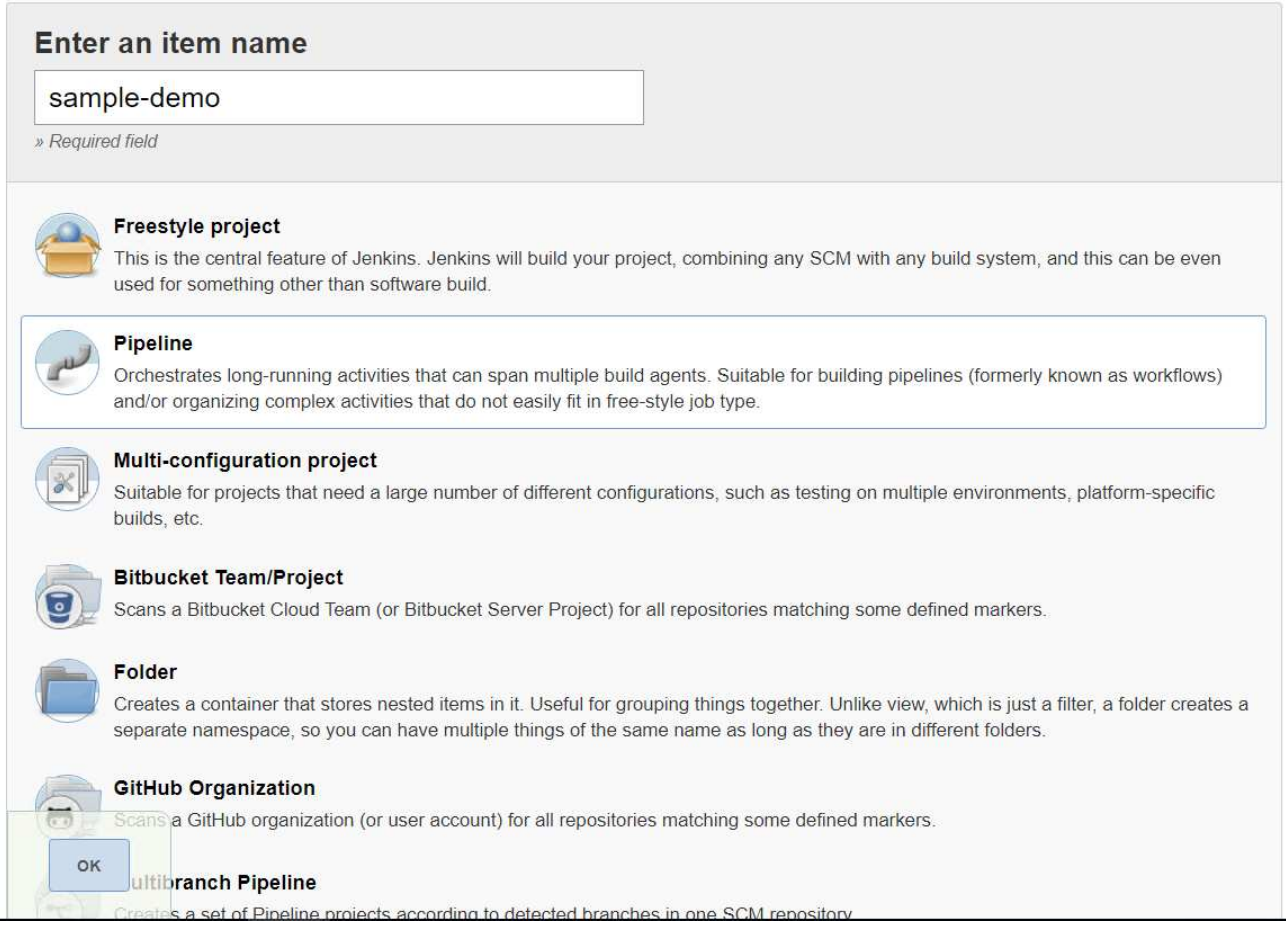
8. Jenkins 시작 페이지가 표시됩니다. Maven 빌드를 사용하기 때문에 Maven 설치를 먼저 완료하십시오. Manage Jenkins > Global Tool Configuration 으로 이동한 다음 Maven 하위 헤드에서 Add Maven 을 클릭합니다. 선택한 이름을 입력하고 자동 설치 옵션이 선택되어 있는지 확인합니다. 저장 을 클릭합니다.



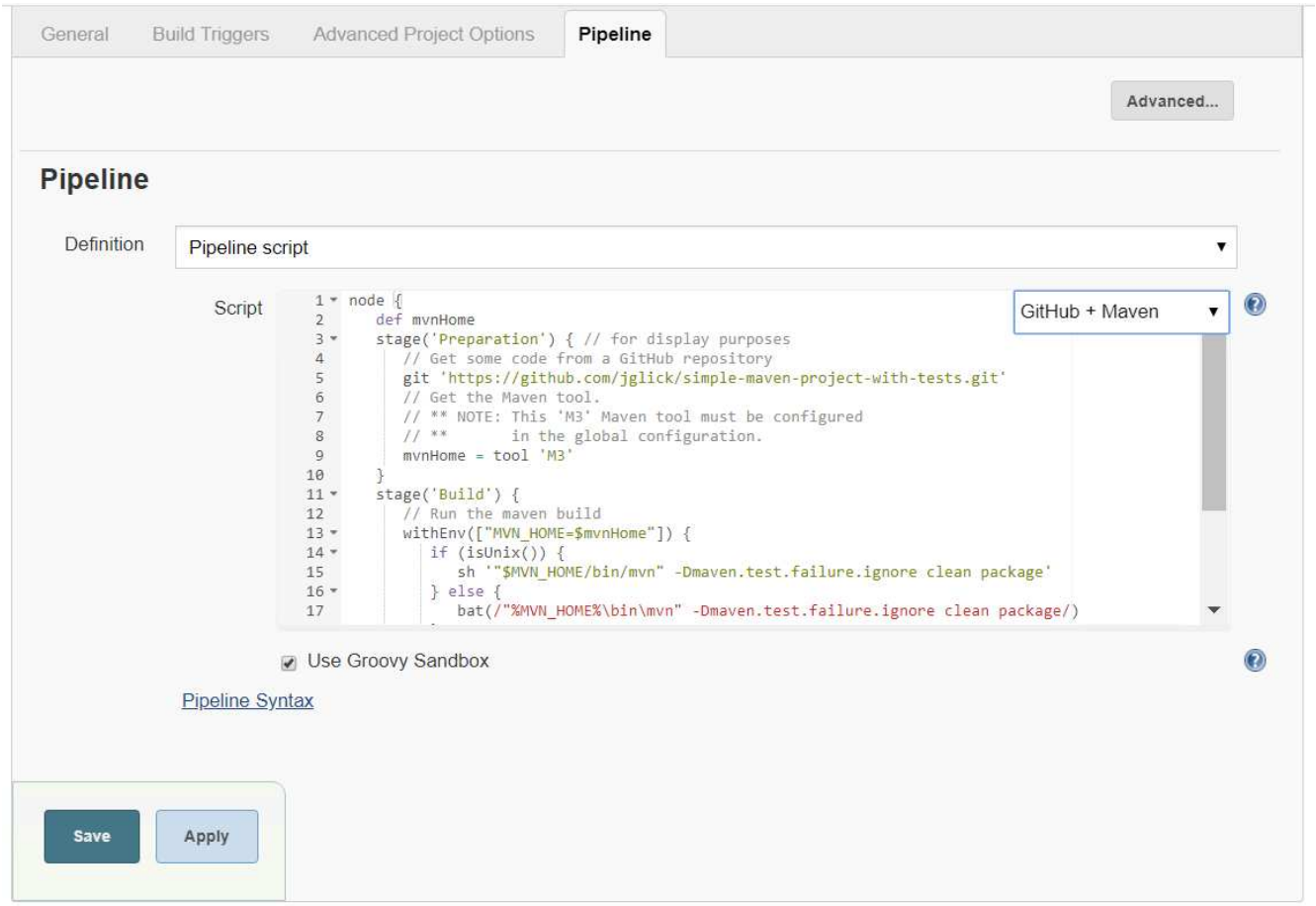
9. 이제 CI/CD 워크플로우를 시연하기 위한 파이프라인을 생성할 수 있습니다. 홈 페이지의 왼쪽 메뉴에서 새 작업 만들기 또는 새 항목 만들기를 클릭합니다.



10. 항목 만들기 페이지에서 선택한 이름을 입력하고 파이프라인을 선택한 다음 확인을 클릭합니다.



11. Pipeline 탭을 선택합니다. 샘플 파이프라인 시험 사용 드롭다운 메뉴에서 GitHub + Maven 을 선택합니다. 코드가 자동으로 채워집니다. 저장 을 클릭합니다.



12. 지금 구축을 클릭하여 준비, 빌드 및 테스트 단계를 통해 개발을 시작합니다. 전체 빌드 프로세스를 완료하고 빌드 결과를 표시하는 데 몇 분 정도 걸릴 수 있습니다.

13. 코드 변경 사항이 있을 때마다 파이프라인을 재구축하여 새로운 소프트웨어 버전을 패치할 수 있으므로 지속적인 통합 및 지속적인 제공이 가능합니다. 최근 변경 내용 을 클릭하여 이전 버전의 변경 내용을 추적합니다.

"다음: 비디오 및 데모."

NetApp ONTAP를 사용하여 Red Hat OpenShift에서 멀티 테넌시를 구성합니다

NetApp과 함께 Red Hat OpenShift에서 멀티 테넌시를 구성합니다

컨테이너에서 여러 애플리케이션 또는 워크로드를 실행하는 많은 조직에서는 애플리케이션 또는 작업 부하당 하나의 Red Hat OpenShift 클러스터를 배포하는 경향이 있습니다. 이를 통해 애플리케이션 또는 워크로드에 대한 엄격한 격리를 구현하고 성능을 최적화하며 보안 취약점을 줄일 수 있습니다. 그러나 각 애플리케이션에 대해 별도의 Red Hat OpenShift 클러스터를 배포하면 자체 문제가 발생합니다. 또한 각 클러스터를 단독으로 모니터링 및 관리해야 하는 운영 오버헤드가 늘어나고, 다양한 애플리케이션을 위한 전용 리소스로 비용이 증가하며, 효율적인 확장성을 저해합니다.

이러한 문제를 해결하기 위해 단일 Red Hat OpenShift 클러스터에서 모든 애플리케이션 또는 워크로드를 실행하는 것을 고려할 수 있습니다. 그러나 이러한 아키텍처에서 리소스 격리 및 애플리케이션 보안 취약점은 주요 과제 중 하나입니다. 한 워크로드의 보안 취약점이 다른 작업 부하로 자연스럽게 유출되어 영향 영역이 증가할 수 있습니다. 또한 한 응용 프로그램에서 갑자기 제어되지 않는 리소스 사용률은 기본적으로 리소스 할당 정책이 없기 때문에 다른 응용 프로그램의 성능에 영향을 줄 수 있습니다.

따라서 조직에서는 단일 클러스터에서 모든 워크로드를 실행하면서 각 워크로드별로 전용 클러스터의 이점을 제공하는

동시에 두 환경 모두에서 최상의 솔루션을 찾는 것이 좋습니다.

이러한 효과적인 솔루션 중 하나는 Red Hat OpenShift에서 멀티 테넌시를 구성하는 것입니다. 멀티 테넌시는 리소스, 보안 등을 적절하게 격리하여 동일한 클러스터에서 여러 테넌트를 함께 사용할 수 있는 아키텍처입니다. 이 컨텍스트에서 테넌트는 특정 사용자 그룹이 단독으로 사용하도록 구성된 클러스터 리소스의 하위 집합으로 볼 수 있습니다. Red Hat OpenShift 클러스터에서 멀티 테넌시를 구성하면 다음과 같은 이점이 있습니다.

- 클러스터 리소스를 공유하여 CapEx 및 OpEx 절감
- 운영 및 관리 오버헤드 감소
- 보안 침해의 교차 오염으로부터 워크로드를 보호합니다
- 리소스 경합으로 인해 예기치 않은 성능 저하로부터 워크로드 보호

완전히 실현된 멀티테넌트 OpenShift 클러스터의 경우 컴퓨팅, 스토리지, 네트워킹, 보안 등과 같은 다양한 리소스 버킷에 속하는 클러스터 리소스에 대해 할당량 및 제한을 구성해야 합니다. 이 솔루션에서 모든 리소스 버킷의 특정 측면을 다루지만, NetApp은 NetApp ONTAP가 지원하는 Astra Trident에 의해 동적으로 할당된 스토리지 리소스에서 멀티테넌시를 구성하여 동일한 Red Hat OpenShift 클러스터에서 여러 워크로드에서 제공 또는 소비되는 데이터를 격리하고 보호하는 모범 사례에 초점을 맞춥니다.

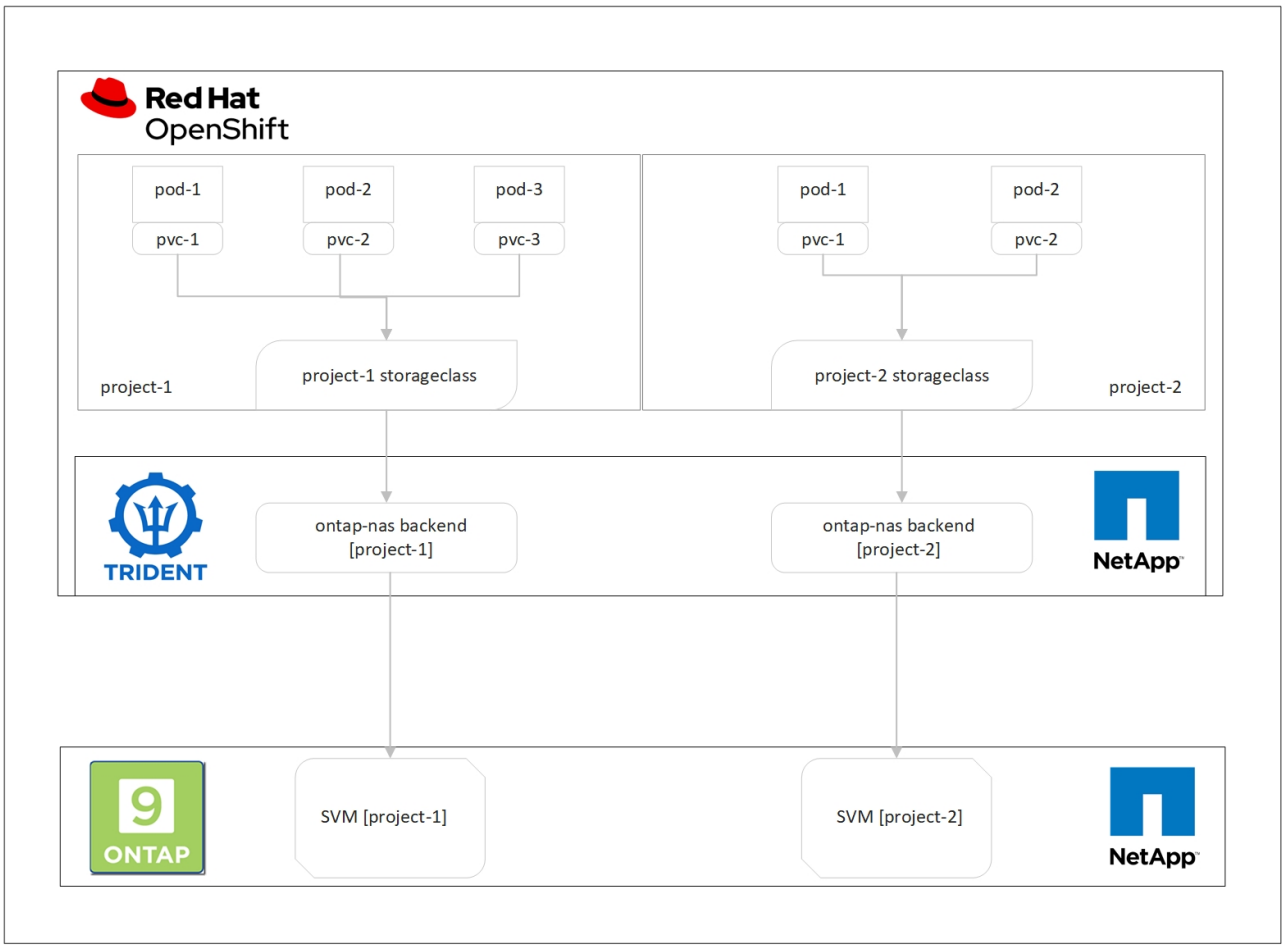
"다음: 아키텍처."

있습니다

NetApp ONTAP에서 지원하는 Red Hat OpenShift 및 Astra Trident는 기본적으로 워크로드 간 격리를 제공하지 않지만 멀티 테넌시를 구성하는 데 사용할 수 있는 다양한 기능을 제공합니다. NetApp ONTAP가 지원하는 Astra Trident가 있는 Red Hat OpenShift 클러스터에서 멀티테넌트 솔루션을 설계하는 방법을 잘 이해하기 위해 일련의 요구 사항이 있는 예제를 고려하고 이 솔루션에 대한 구성을 개략적으로 설명합니다.

한 조직이 Red Hat OpenShift 클러스터에서 두 개의 다른 팀이 진행 중인 두 프로젝트의 일부로 두 개의 워크로드를 실행하는 것으로 가정해 보겠습니다. 이러한 워크로드의 데이터는 NetApp ONTAP NAS 백엔드의 Astra Trident가 동적으로 프로비저닝한 PVC에 있습니다. 조직은 이러한 두 워크로드를 위한 멀티테넌트 솔루션을 설계하고 이러한 프로젝트에 사용되는 리소스를 격리하여 보안 및 성능이 유지되도록 해야 합니다. 이러한 애플리케이션은 주로 해당 애플리케이션을 지원하는 데이터에 초점을 맞춥니다.

다음 그림은 NetApp ONTAP가 지원하는 Astra Trident가 있는 Red Hat OpenShift 클러스터의 멀티 테넌트 솔루션을 보여줍니다.



기술 요구 사항

1. NetApp ONTAP 스토리지 클러스터
2. Red Hat OpenShift 클러스터
3. 아스트라 트리덴트

Red Hat OpenShift – 클러스터 리소스

Red Hat OpenShift 클러스터의 관점에서 볼 때 가장 먼저 해야 할 리소스는 프로젝트입니다. OpenShift 프로젝트는 전체 OpenShift 클러스터를 여러 가상 클러스터로 분할하는 클러스터 리소스로 볼 수 있습니다. 따라서 프로젝트 수준에서 격리하면 다중 임차를 구성할 수 있는 기반이 제공됩니다.

다음으로, 클러스터에서 RBAC를 구성합니다. 가장 좋은 방법은 모든 개발자가 IDP(Identity Provider)에서 단일 프로젝트 또는 작업 부하를 단일 사용자 그룹으로 구성하도록 하는 것입니다. Red Hat OpenShift는 IDP 통합 및 사용자 그룹 동기화를 지원하므로 IDP의 사용자 및 그룹을 클러스터로 가져올 수 있습니다. 이렇게 하면 클러스터 관리자가 프로젝트 전용 클러스터 리소스의 액세스를 해당 프로젝트에서 작업하는 사용자 그룹 또는 그룹으로 분리하여 모든 클러스터 리소스에 대한 무단 액세스를 제한할 수 있습니다. Red Hat OpenShift와의 IDP 통합에 대한 자세한 내용은 설명서를 참조하십시오 "여기".

NetApp ONTAP를 참조하십시오

각 프로젝트에 대해 스토리지에서 생성된 볼륨이 별도의 스토리지에 생성된 것처럼 호스트에 나타나도록 하기 위해 Red

Hat OpenShift 클러스터의 영구 스토리지 공급자 역할을 하는 공유 스토리지를 격리하는 것이 중요합니다. 이를 위해 프로젝트나 작업 부하가 있는 것처럼 NetApp ONTAP에 SVM(스토리지 가상 시스템)을 최대한 많이 생성하고 각 SVM을 작업 부하에 전용으로 사용합니다.

아스트라 트리덴트

NetApp ONTAP에서 생성된 다양한 프로젝트에 대해 서로 다른 SVM을 생성한 후 각 SVM을 다른 Trident 백엔드에 매핑해야 합니다. Trident의 백엔드 구성은 OpenShift 클러스터 리소스에 영구 스토리지를 할당하는 역할을 하므로 SVM에 매핑할 세부 정보가 필요합니다. 이 드라이버는 최소한 백엔드의 프로토콜 드라이버여야 합니다. 선택적으로, 볼륨에 스토리지의 용량을 할당하는 방법을 정의하고 볼륨 크기 또는 애그리게이트 사용량 등에 대한 제한을 설정할 수 있습니다. Trident 백엔드의 정의와 관련된 세부 정보를 찾을 수 있습니다 "여기".

Red Hat OpenShift – 스토리지 리소스

Trident 백엔드를 구성한 후 다음 단계는 StorageClasses를 구성하는 것입니다. 백엔드가 있는 만큼 스토리지 클래스를 구성하여 각 스토리지 클래스 액세스 권한을 통해 백엔드에서 볼륨을 한 개만 스핀업할 수 있습니다. 스토리지 클래스를 정의하는 동안 storagePools 매개 변수를 사용하여 StorageClass를 특정 Trident 백엔드에 매핑할 수 있습니다. 스토리지 클래스를 정의하는 세부 정보를 찾을 수 있습니다 "여기". 따라서 StorageClass에서 Trident 백엔드로 일대일로 매핑하여 하나의 SVM을 가리키도록 합니다. 이렇게 하면 해당 프로젝트에 할당된 StorageClass를 통해 모든 스토리지 청구가 해당 프로젝트 전용 SVM에서 서비스됩니다.

스토리지 클래스는 이름이 같은 리소스가 아니므로 다른 네임스페이스나 프로젝트의 Pod를 통해 한 프로젝트의 스토리지 클래스에 대한 스토리지 클레임이 거부되도록 하려면 어떻게 해야 합니까? 리소스 할당량을 사용하면 됩니다. ResourceQuotas 는 프로젝트당 리소스의 총 사용량을 제어하는 개체입니다. 프로젝트의 개체에서 사용할 수 있는 총 리소스 양뿐만 아니라 수를 제한할 수 있습니다. 리소스 할당량을 사용하면 프로젝트의 거의 모든 리소스를 제한할 수 있으며, 이러한 리소스를 효율적으로 사용하면 리소스 오버 프로비저닝 또는 과소비로 인한 비용 및 운영 중단을 줄일 수 있습니다. 설명서를 참조하십시오 "여기" 를 참조하십시오.

이 활용 사례에서는 특정 프로젝트의 포드를 해당 프로젝트 전용이 아닌 스토리지 클래스에서 스토리지를 청구하는 것을 제한해야 합니다. 이렇게 하려면 "<storage-class-name>.storageclass.storage.k8s.io/persistentvolumeclaims"를 0으로 설정하여 다른 스토리지 클래스에 대한 영구 볼륨 클레임을 제한해야 합니다. 또한 클러스터 관리자는 프로젝트의 개발자가 리소스 할당량을 수정할 수 있는 액세스 권한이 없어야 합니다.

"다음: 구성."

구성

모든 멀티테넌트 솔루션의 경우 사용자는 필요한 것보다 더 많은 클러스터 리소스에 액세스할 수 없습니다. 따라서 다중 테넌시 구성의 일부로 구성할 전체 리소스 세트는 클러스터 관리자, 스토리지 관리자 및 각 프로젝트를 작업하는 개발자가 구성합니다.

다음 표에는 여러 사용자가 수행해야 하는 여러 작업이 정리되어 있습니다.

역할	작업
* 클러스터 관리자 *	다양한 애플리케이션 또는 워크로드를 위한 프로젝트를 생성합니다
	storage-admin에 대한 ClusterRoles 및 RoleBindings를 생성합니다
	특정 프로젝트에 대한 액세스를 할당하는 개발자를 위한 역할 및 RoleBindings를 만듭니다
	[선택 사항] 특정 노드에서 Pod를 예약하도록 프로젝트를 구성합니다
* 스토리지 - 관리자 *	NetApp ONTAP에서 SVM을 생성합니다
	Trident 백엔드를 생성합니다
	StorageClasses를 생성합니다
	스토리지 리소스 할당량을 생성합니다
개발자 *	할당된 프로젝트에서 PVC 또는 POD를 생성하거나 패치하기 위한 액세스를 검증합니다
	다른 프로젝트에서 PVC 또는 POD를 생성하거나 패치하기 위한 액세스를 검증합니다
	프로젝트, 리소스 할당량 및 StorageClasses를 보거나 편집하려면 액세스 유효성을 검사합니다

"다음: 필수 구성 요소."

구성

필수 구성 요소

- NetApp ONTAP 클러스터
- Red Hat OpenShift 클러스터
- 클러스터에 설치된 Trident
- tridentctl 및 OC 도구가 설치되고 $PATH에 추가된 관리 워크스테이션
- ONTAP에 대한 관리자 액세스
- OpenShift 클러스터에 대한 클러스터 관리자 액세스
- 클러스터가 ID 공급자와 통합됩니다
- ID 공급자는 서로 다른 팀의 사용자를 효율적으로 구분하도록 구성되어 있습니다

"다음: 클러스터 관리자 작업."

구성: 클러스터 관리 작업

Red Hat OpenShift cluster-admin은 다음과 같은 작업을 수행합니다.

1. Red Hat OpenShift 클러스터에 cluster-admin으로 로그인합니다.

2. 서로 다른 프로젝트에 해당하는 두 개의 프로젝트를 작성합니다.

```
oc create namespace project-1
oc create namespace project-2
```

3. PROJECT-1의 개발자 역할을 만듭니다.

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-1
  name: developer-project-1
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
```

```
      - podtemplates
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

> 이 섹션에 제공된 역할 정의는 예일 뿐입니다. 개발자 역할은 최종 사용자 요구 사항에 따라 정의되어야 합니다.

1. 마찬가지로 project-2에 대한 개발자 역할을 만듭니다.
2. 모든 OpenShift 및 NetApp 스토리지 리소스는 일반적으로 스토리지 관리자가 관리합니다. 스토리지 관리자를 위한 액세스는 Trident가 설치될 때 생성되는 덴트 운영자 역할에 의해 제어됩니다. 또한 스토리지 관리자는 리소스 할당량에 대한 액세스 권한이 있어야 스토리지 소비 방식을 제어할 수 있습니다.
3. 클러스터의 모든 프로젝트에서 리소스 할당량을 관리하는 역할을 생성하여 스토리지 관리자에게 연결합니다.

```
cat << EOF | oc create -f -
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: resource-quotas-role
rules:
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - resourcequotas
  - verbs:
      - '*'
    apiGroups:
      - quota.openshift.io
    resources:
      - '*'
EOF
```

4. 클러스터가 조직의 ID 공급자와 통합되었으며 사용자 그룹이 클러스터 그룹과 동기화되었는지 확인합니다. 다음 예제에서는 ID 공급자가 클러스터와 통합되고 사용자 그룹과 동기화되었음을 보여 줍니다.

```
$ oc get groups
NAME                          USERS
ocp-netapp-storage-admins     ocp-netapp-storage-admin
ocp-project-1                 ocp-project-1-user
ocp-project-2                 ocp-project-2-user
```

1. 스토리지 관리자용 ClusterRoleBindings를 구성합니다.

```
cat << EOF | oc create -f -
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-trident-operator
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: trident-operator
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-resource-quotas-cr
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: resource-quotas-role
EOF
```

ⓘ 스토리지 관리자의 경우 세 가지 역할, 즉 세 가지 운영자 및 리소스 할당량이 바인딩되어야 합니다.

1. Developer-project-1 역할을 project-1의 해당 그룹(OCP-project-1)에 바인딩하는 개발자를 위한 RoleBindings를 만듭니다.

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-1-developer
  namespace: project-1
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-1
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-1
EOF
```

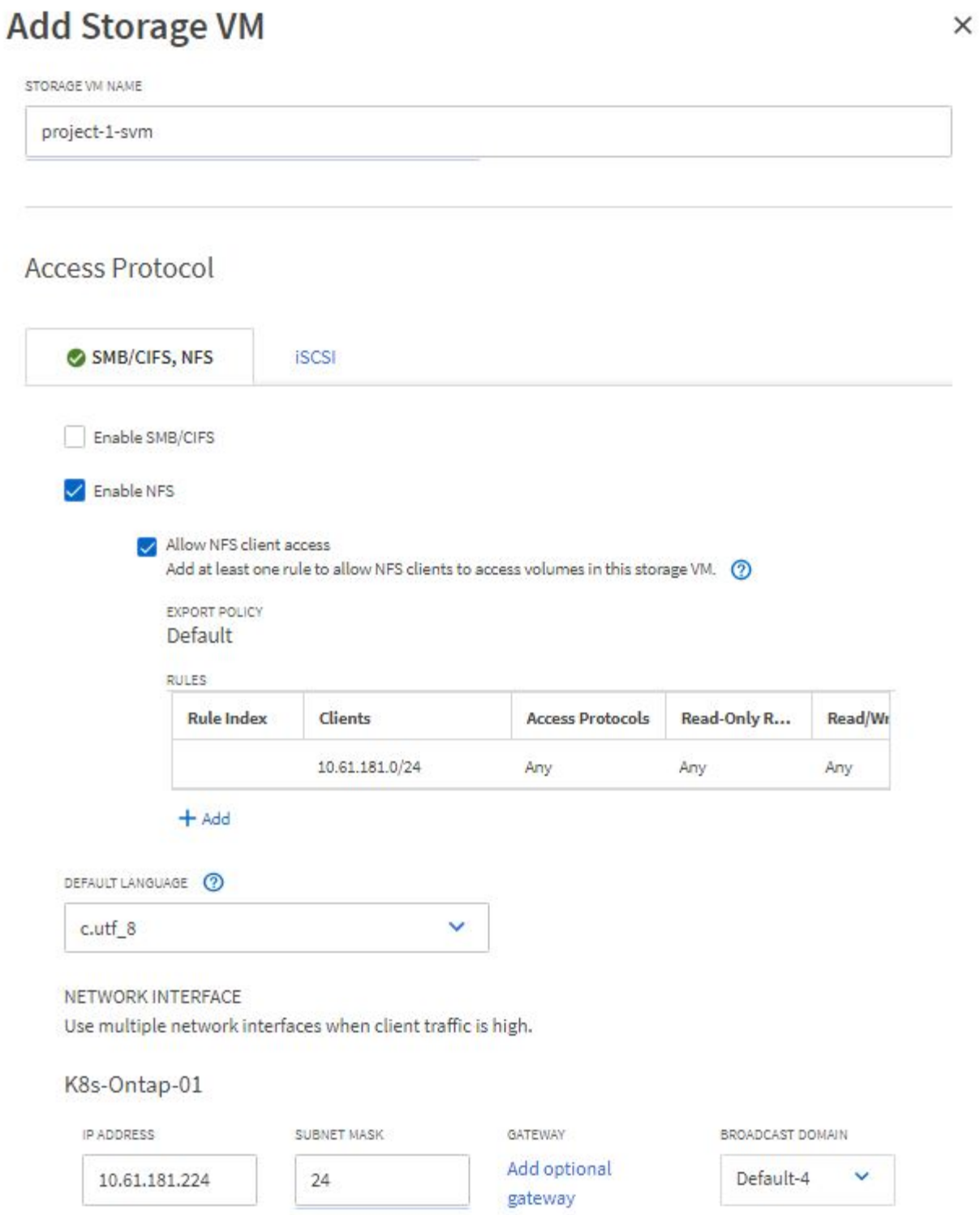
2. 마찬가지로 개발자 역할을 프로젝트 2의 해당 사용자 그룹에 바인딩하는 개발자를 위한 RoleBindings 를 만듭니다.

"다음: 스토리지 관리자 작업."

구성: 스토리지 관리 작업

스토리지 관리자가 다음 리소스를 구성해야 합니다.

1. NetApp ONTAP 클러스터에 admin으로 로그인합니다.
2. Storage > Storage VMs 로 이동하고 Add 를 클릭합니다. 필요한 세부 정보를 제공하여 프로젝트-1과 프로젝트-2에 각각 필요한 SVM 2개를 생성합니다. SVM 및 리소스를 관리하는 vsadmin 계정도 생성합니다.



1. 스토리지 관리자로 Red Hat OpenShift 클러스터에 로그인합니다.
2. 프로젝트-1의 백엔드를 생성하여 프로젝트 전용 SVM에 매핑합니다. ONTAP 클러스터 관리자를 사용하는 대신 SVM의 vsadmin 계정을 사용하여 백엔드를 SVM에 연결하는 것이 좋습니다.

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_1",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.224",
    "svm": "project-1-svm",
    "username": "vsadmin",
    "password": "NetApp123"
}
EOF
```

이 예에서는 ONTAP-NAS 드라이버를 사용하고 있습니다. 사용 사례에 따라 백엔드를 생성할 때 적절한 드라이버를 사용하십시오.

트리덴트 프로젝트에 Trident가 설치되어 있다고 가정합니다.

1. 마찬가지로, project-2를 위한 Trident 백엔드를 생성한 다음 project-2 전용 SVM에 매핑합니다.
2. 그런 다음 스토리지 클래스를 생성합니다. storagePools 매개 변수를 설정하여 project-1용 스토리지 클래스를 생성하고 backend 전용 스토리지 풀을 project-1에 사용하도록 구성합니다.

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-1-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_1:.*"
EOF
```

3. 마찬가지로, project-2에 대한 스토리지 클래스를 생성하고 백엔드에서 project-2에 대한 스토리지 풀을 사용하도록 구성합니다.
4. ResourceQuota를 생성하여 다른 프로젝트 전용 스토리지로부터 스토리지를 요청하는 Project-1의 리소스를 제한합니다.

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-1-sc-rq
  namespace: project-1
spec:
  hard:
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

5. 마찬가지로, ResourceQuota를 생성하여 다른 프로젝트 전용 스토리지로부터 스토리지를 요청하는 Project-2의 리소스를 제한합니다.

"다음: 검증."

검증

이전 단계에서 구성한 멀티테넌트 아키텍처를 확인하려면 다음 단계를 수행하십시오.

할당된 프로젝트에서 **PVC** 또는 **POD**를 생성하기 위한 액세스를 검증합니다

1. PROJECT-1의 개발자인 OCP-PROJECT-1-USER로 로그인합니다.
2. 새 프로젝트를 만들려면 액세스 권한을 확인하십시오.

```
oc create ns sub-project-1
```

3. PROJECT-1에 할당된 storageclass를 사용하여 PROJECT-1에서 PVC를 생성한다.

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. PVC와 관련된 PV를 확인한다.

```
oc get pv
```

5. PV 및 해당 볼륨이 NetApp ONTAP의 PROJECT-1 전용 SVM에서 생성되었는지 확인합니다.

```
volume show -vserver project-1-svm
```

6. PROJECT-1에서 POD를 생성하고 이전 단계에서 만든 PVC를 마운트합니다.

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  volumes:
    - name: test-pvc-project-1
      persistentVolumeClaim:
       claimName: test-pvc-project-1
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: test-pvc-project-1
EOF
```

7. POD가 실행 중이고 볼륨이 마운트되어 있는지 확인합니다.

```
oc describe pods test-pvc-pod -n project-1
```

다른 프로젝트에서 **PVC** 또는 **POD**를 생성하거나 다른 프로젝트 전용 리소스를 사용하도록 액세스를 검증합니다

1. PROJECT-1의 개발자인 OCP-PROJECT-1-USER로 로그인합니다.
2. PROJECT-2에 할당된 storageclass를 사용하여 PROJECT-1에서 PVC를 생성한다.

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1-sc-2
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-2-sc
EOF
```

3. PROJECT-2에서 PVC를 작성합니다.

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-2-sc-1
  namespace: project-2
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. PVC의 TEST-PVC-PROJECT-1-SC-2, TEST-PVC-PROJECT-2-SC-1이 생성되지 않았는지 확인한다.

```
oc get pvc -n project-1
oc get pvc -n project-2
```

5. PROJECT-2에서 POD를 작성합니다.

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
EOF
```

프로젝트, 리소스 할당량 및 **StorageClasses**를 보고 편집하려면 액세스 권한을 확인합니다

1. PROJECT-1의 개발자인 OCP-PROJECT-1-USER로 로그인합니다.
2. 새 프로젝트를 만들려면 액세스 권한을 선택합니다.

```
oc create ns sub-project-1
```

3. 프로젝트 보기에 대한 액세스 권한 확인

```
oc get ns
```

4. 사용자가 PROJECT-1에서 ResourceQuotas를 보거나 편집할 수 있는지 확인합니다.

```
oc get resourcequotas -n project-1
oc edit resourcequotas project-1-sc-rq -n project-1
```

5. 사용자가 스토리지 시스템을 볼 수 있는 액세스 권한이 있는지 확인합니다.

```
oc get sc
```

6. 스토리지 풀 설명을 위한 액세스를 확인하십시오.
7. 스토리지 풀 편집을 위한 사용자 액세스 권한 검증

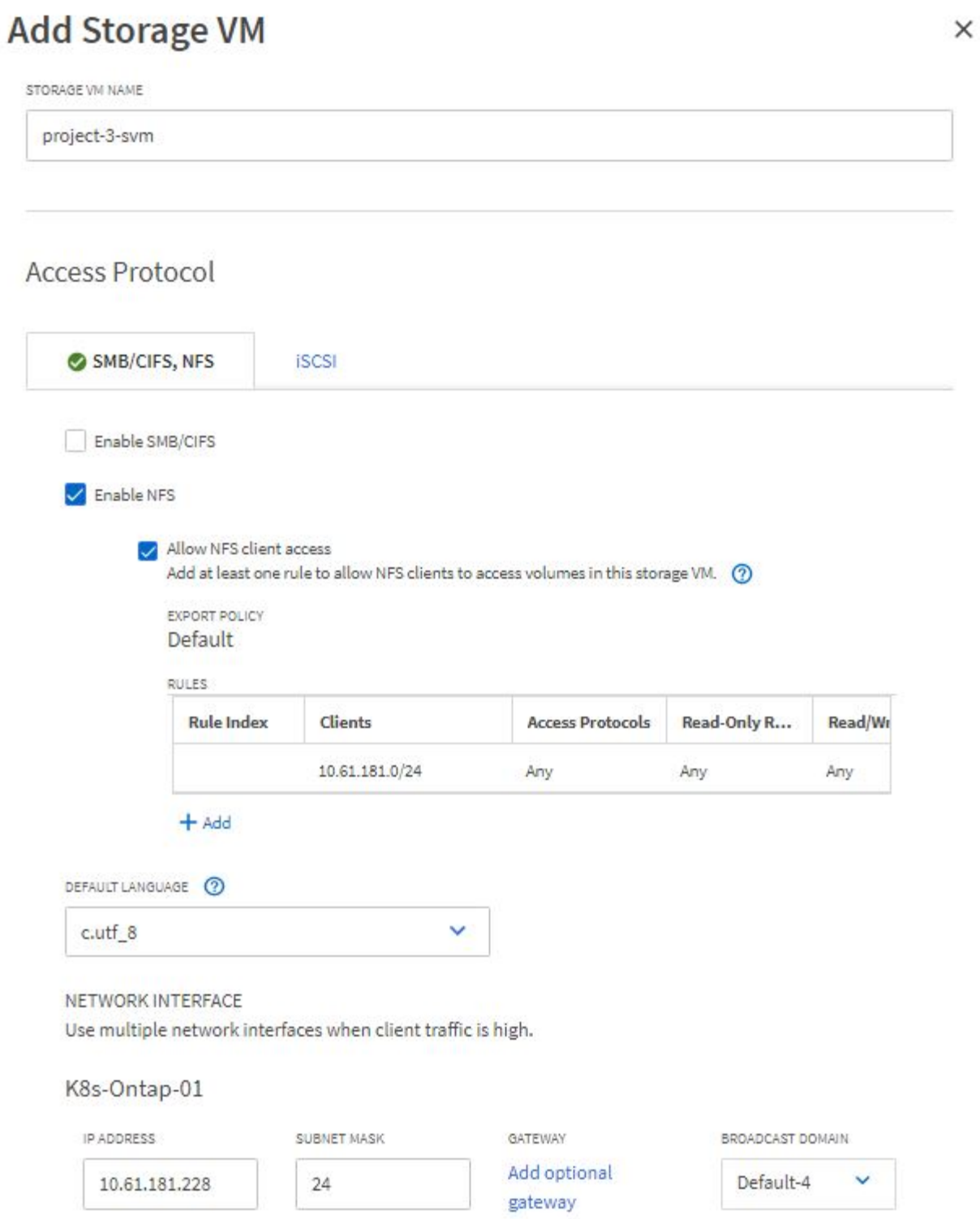
```
oc edit sc project-1-sc
```

"다음: 확장."

확장**:** 더 많은 프로젝트 추가

멀티 테넌트 구성에서는 스토리지 리소스로 새 프로젝트를 추가하려면 멀티 테넌시가 위반되지 않도록 추가적인 구성이 필요합니다. 멀티 테넌트 클러스터에 프로젝트를 더 추가하려면 다음 단계를 완료하십시오.

1. NetApp ONTAP 클러스터에 스토리지 관리자로 로그인합니다.
2. 스토리지 → 스토리지 VM으로 이동한 후 추가 를 클릭합니다. PROJECT-3 전용 SVM을 새로 생성합니다. SVM 및 리소스를 관리하는 vsadmin 계정도 생성합니다.



1. Red Hat OpenShift 클러스터에 클러스터 관리자로 로그인합니다.
2. 새 프로젝트를 만듭니다.

```
oc create ns project-3
```

3. PROJECT-3의 사용자 그룹이 IDP에서 생성되고 OpenShift 클러스터와 동기화되었는지 확인합니다.

```
oc get groups
```

4. PROJECT-3의 개발자 역할을 만듭니다.

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-3
  name: developer-project-3
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
      - podtemplates
```

```
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

> 이 섹션에 제공된 역할 정의는 예일 뿐입니다. 개발자 역할은 최종 사용자 요구 사항에 따라 정의되어야 합니다.

1. 개발자를 위한 RoleBinding 만들기 project-3에서 developer-project-3 역할을 프로젝트-3의 해당 그룹(OCP-PROJECT-3)에 바인딩합니다.

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-3-developer
  namespace: project-3
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-3
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-3
EOF
```

2. Red Hat OpenShift 클러스터에 스토리지 관리자로 로그인합니다
3. Trident 백엔드를 생성하여 PROJECT-3 전용 SVM에 매핑합니다. ONTAP 클러스터 관리자를 사용하는 대신 SVM의 vsadmin 계정을 사용하여 백엔드를 SVM에 연결하는 것이 좋습니다.

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_3",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.228",
    "svm": "project-3-svm",
    "username": "vsadmin",
    "password": "NetApp!23"
}
EOF
```

이 예에서는 ONTAP-NAS 드라이버를 사용하고 있습니다. 사용 사례에 따라 백엔드를 생성하는 데 적합한 드라이버를 사용하십시오.

트리덴트 프로젝트에 Trident가 설치되어 있다고 가정합니다.

1. PROJECT-3의 스토리지 클래스를 생성하고 PROJECT-3 전용 백엔드에서 스토리지 풀을 사용하도록 구성합니다.

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-3-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_3:.*"
EOF
```

2. ResourceQuota를 생성하여 다른 프로젝트 전용 스토리지로부터 스토리지를 요청하는 Project-3의 리소스를 제한합니다.

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-3-sc-rq
  namespace: project-3
spec:
  hard:
    project-1-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

3. 다른 프로젝트의 ResourceQuotas를 패치하여 해당 프로젝트의 리소스가 project-3 전용 스토리지 클래스에서 스토리지에 액세스하는 것을 제한합니다.

```
oc patch resourcequotas project-1-sc-rq -n project-1 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
oc patch resourcequotas project-2-sc-rq -n project-2 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
```

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.